

Project ClearCase

Offline-First, Voice-Native AI Dispute Resolution Mesh for Bharat

Hackathon: Bharat Builds by WeMakeDevs

1. Problem Statement (PS)

Over 4 crore civil cases are pending in Indian courts, and a huge share never even reach a court — they die out informally at the village level because:

- Access barrier: Formal legal process is slow, expensive, and intimidating for rural citizens.
- Literacy & language barrier: Legal text is in English/formal Hindi legalese; most disputants speak regional dialects (Haryanvi, Bhojpuri, Awadhi, Punjabi, etc.) and can't read at all.
- Knowledge void at the grassroots: Panchayats and local mediators resolve disputes from memory or social pressure, not from the actual Land Revenue Act / Shops & Establishment Act / Minimum Wages Act clause that applies — leading to inconsistent, biased, or unenforceable outcomes.
- No paper trail: Verbal panchayat settlements have no tamper-proof record, so they're reopened, denied, or exploited later.
- Connectivity gap: Much of rural Bharat has patchy internet and a large population still on feature phones (2G/IVR/SMS), which most legal-tech products ignore entirely.

Core problem statement: *How do we give an illiterate, dialect-speaking villager the same quality of legal grounding a city lawyer has access to — instantly, in their own voice, with a tamper-proof outcome — without requiring smartphones, literacy, or internet reliability?*

2. Solution Overview

ClearCase is an agentic AI mediation layer that sits between citizens and local adjudicators (Panchayats / Lok Adalats / NGOs). A citizen simply speaks their grievance in their own dialect.

The system:

1. Transcribes & translates the grievance.
2. Retrieves the exact relevant law/clause from a state-specific legal knowledge base (RAG over OpenSearch).
3. Uses an LLM (Bedrock/Claude) to draft a neutral, plain-language settlement proposal and a chronological case file.
4. Reads the draft back out loud in the citizen's own language (TTS) so literacy is never a blocker.
5. Once both parties verbally/OTP-confirm agreement, anchors a cryptographic hash of the settlement on-chain so it can never be silently altered or denied later.
6. If the AI is not confident, or the dispute is legally complex, it escalates to a human adjudicator instead of forcing an answer — the system knows its own limits.

It is explicitly not trying to replace courts or give binding legal judgments — it produces non-binding neutral settlement drafts that speed up voluntary, informed resolution, exactly the role Panchayats/Lok Adalats already play, just with actual legal grounding behind it.

3. Refined & Expanded Feature Set

Tier 1 — Core (must-build for demo)

1. Voice-first grievance intake — record in any regional dialect, one big mic button.
2. Multi-agent RAG pipeline — Transcription Agent → Statutory Matching Agent → Mediation Draft Agent.
3. State-specific legal knowledge base (start with 1 state, e.g. UP or Haryana Land Revenue Act + Minimum Wages Act).
4. Structured output: Grievance Summary → Applicable Section (with citation) → Suggested Resolution Draft.
5. Voice playback (TTS) of the draft in the same regional language — closes the literacy loop.
6. On-chain settlement hash anchoring (Polygon Amoy testnet) once both parties confirm.
7. Case Dashboard for the Panchayat/mediator showing grievance, cited law, draft, and status.

Tier 2 — Differentiators (huge score in judging, feasible in 24h with mocks)

8. Confidence & escalation engine — if statutory match confidence is low or dispute type is out-of-scope (e.g. criminal, matrimonial), the AI flags "Human Mediator Review Required" instead of guessing. This is your biggest trust/safety feature.
9. IVR / SMS fallback mode — a Twilio/Exotel-style stub showing how a feature-phone user could call a number, speak after a beep, and later get an SMS with the settlement summary in their language. (Even a mocked flow diagram + one working IVR call demonstrates Bharat-scale thinking.)
10. Dual-party OTP e-consent — both disputants must OTP-confirm before the hash is written on-chain, simulating informed consent (not just one party's word).
11. Offline-first PWA — service worker caches the UI shell and queues audio recordings locally, auto-syncing when connectivity returns (core "offline-first" claim from your original pitch — make it real, even minimally).
12. Bilingual/multilingual toggle — English + at least 2 regional languages in the UI text itself, not just voice.
13. Case timeline / audit trail view — chronological, tamper-evident log of every step (recorded → matched → drafted → confirmed → anchored), pulled from DynamoDB + chain hash.

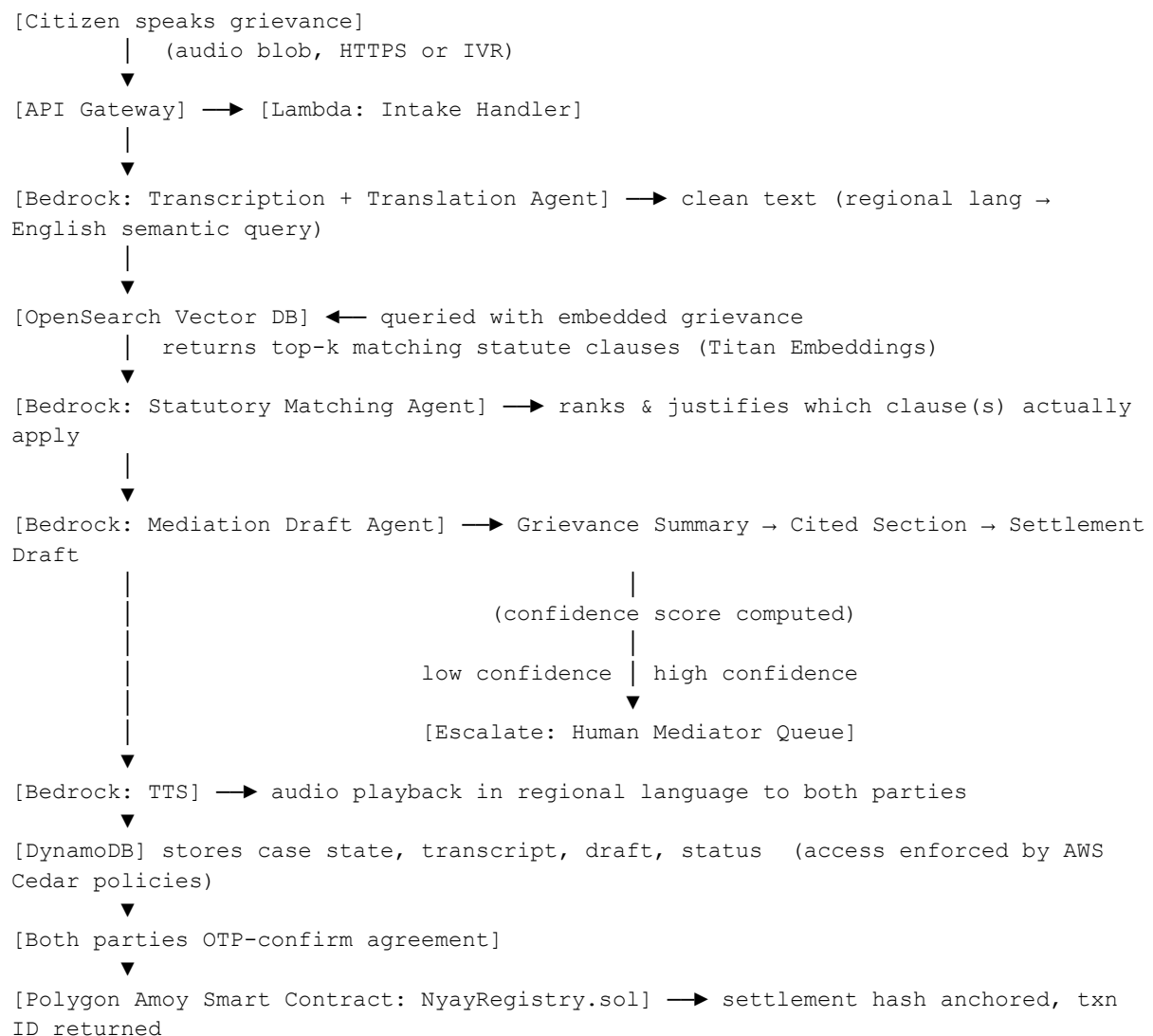
Tier 3 — Stretch (mention as roadmap if time-constrained)

14. Panchayat leaderboard/analytics — anonymized dashboard for state legal-aid bodies showing dispute-type trends by district (policy-value story).
15. Precedent memory — similar past resolved disputes (vector-searched) surfaced to the mediator as reference, building institutional memory villages currently lack.
16. Formal court hand-off packet — one-click export of the full case file as a court-ready PDF if a party rejects the settlement and wants to escalate formally.
17. WhatsApp Business API bot as an alternative low-friction entry point (very high WhatsApp penetration in rural India).
18. Aadhaar-lite identity binding (mocked) — ties the settlement hash to verified party identities without storing raw Aadhaar data (privacy-by-design).

4. User Flow

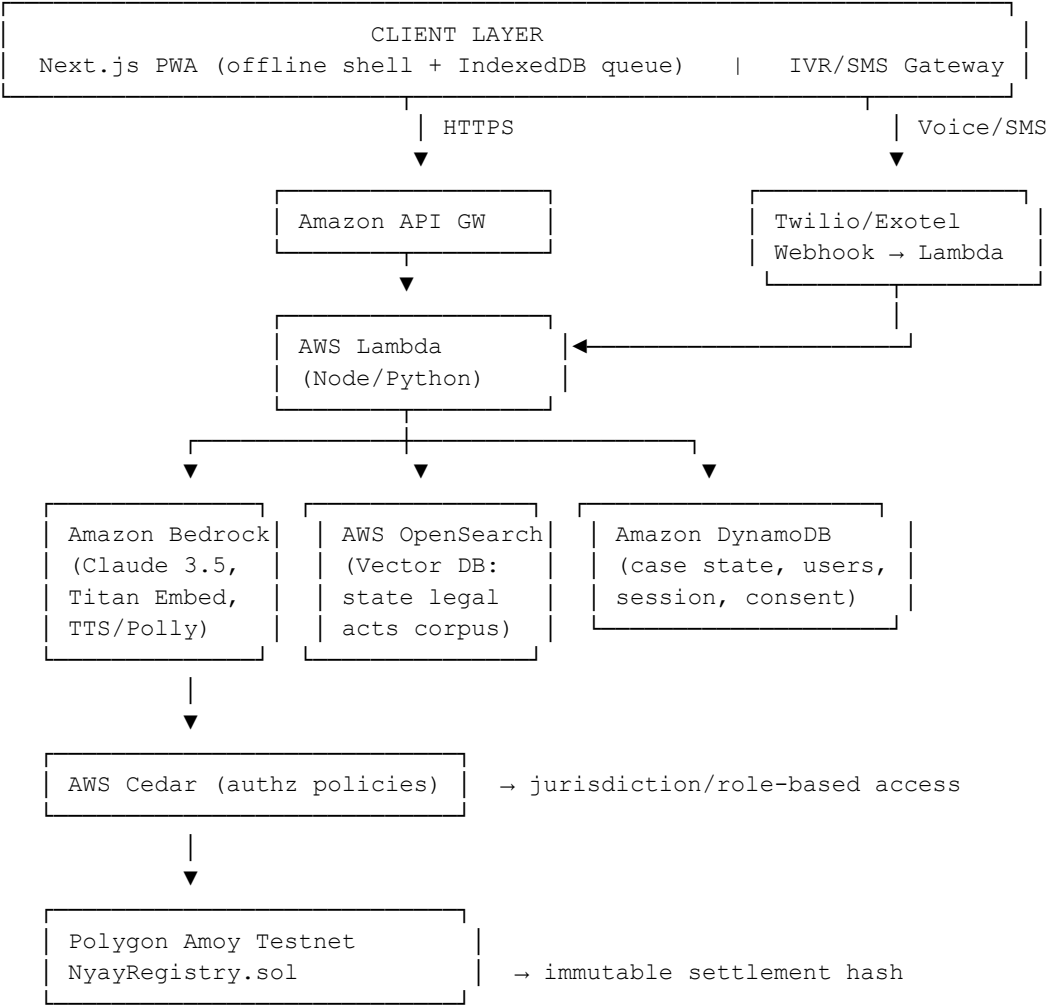
1. Citizen A opens the web app (or calls IVR number) → selects language → taps mic → speaks grievance ("Mera padosi meri zameen ki seema paar kar raha hai...").
2. App shows a live waveform + "Listening..." (accessibility cue for low-literacy users, no need to read anything).
3. Backend transcribes + translates → shows a simple confirmation screen ("Yeh aapne kaha: ...") read aloud via TTS for the citizen to confirm.
4. System asks for Citizen B's (the other party's) phone number to invite them into the same case (OTP-based join).
5. Both grievances (if B also responds) are merged into one case file.
6. AI retrieves matching law sections, drafts a neutral settlement → read aloud in both languages to both parties.
7. Each party taps/says "I Agree" (OTP-confirmed) or "I Disagree, escalate to Panchayat."
8. On mutual agreement → settlement hash written on-chain, confirmation SMS/voice-note sent to both.
9. On disagreement or low-confidence case → routed to a Panchayat Mediator Dashboard where a human reviews the AI draft, edits it, and the same on-chain confirmation flow applies after human sign-off.

5. Execution Flow (System Pipeline)



▼
[Case marked RESOLVED] → confirmation SMS/voice-note + downloadable case file PDF

6. Tech Architecture & System Design



Infra: Docker + AWS SAM/Terraform, deployed via CI/CD, observed via CloudWatch.

7. Tech Stack Summary

Layer	Stack
Frontend	Next.js, React, Tailwind CSS, Web Audio API, Lucide Icons, PWA/Service Worker
Voice Fallback	Twilio/Exotel IVR, SMS webhook (mocked/stub acceptable for demo)
Backend/API	Node.js or Python FastAPI, AWS Lambda, Amazon API Gateway

AI/ML Orchestration	LangChain or AutoGen, Amazon Bedrock (Claude 3.5 Sonnet, Titan Embeddings, Polly/Bedrock TTS)
Vector DB	AWS OpenSearch Service
State/Session DB	Amazon DynamoDB
AuthZ	AWS Cedar (fine-grained, jurisdiction-based policies)
Blockchain	Solidity smart contract () on Polygon Amoy testnet, ethers.js/web3.js
Infra/DevOps	Docker, AWS SAM or Terraform, IAM, CloudWatch, GitHub Actions CI/CD

8. What Makes This Unique

- Voice-first, not text-first legal AI — almost every legal-tech hackathon project assumes literacy and a chatbot UI. ClearCase assumes neither.
 - Confidence-aware escalation — it doesn't try to "solve" every dispute with AI; it knows when to hand off to a human, which is the difference between a toy demo and something a real Panchayat could trust.
 - Grounded, cited legal reasoning (RAG), not hallucinated advice — every suggestion is traceable to an actual statute section, addressing the single biggest trust problem with LLMs in legal contexts.
 - Designed for the actual connectivity/device reality of rural Bharat — offline-first PWA + IVR/SMS fallback, not just "mobile-responsive."
 - Tamper-proof but non-binding — it strengthens informal, community-level dispute resolution (which India already relies on at massive scale) rather than trying to replace the judiciary, making adoption realistic.
-

9. Potential Impact on Bharat

- Judicial backlog relief: even deflecting 10–15% of low-value civil disputes away from formal courts at the pre-litigation stage would meaningfully reduce pendency over time.
 - Access to justice for the literacy-excluded: voice-in, voice-out design reaches the ~25%+ of India's rural population with low/no literacy, who are currently locked out of both formal courts and text-based legal-tech.
 - Consistency & fairness at the grassroots: Panchayat decisions grounded in actual statutes rather than social pressure reduce caste/gender/power-imbalance bias in informal adjudication.
 - Trust infrastructure for informal settlements: blockchain-anchored hashes give villagers something formal courts already have — a settlement that can't be quietly denied later — without needing formal court involvement.
 - Scalable government tooling: the anonymized dispute-analytics layer (Tier 3) gives state legal-aid authorities real visibility into where and why disputes cluster, informing policy (e.g., where land record digitization is most urgently needed).
-

10. 24-Hour Execution Roadmap

Time	AI/ML	Backend	Frontend	DevOps/Blockchain
0–4h	Curate 3–5 legal PDFs, clean to text, set up Bedrock client	API Gateway + Lambda skeleton, DynamoDB schema	Next.js boilerplate, mic recording hook	IAM roles, OpenSearch cluster, repo/CI skeleton
5–12h	LangChain agent chain (transcribe → match → draft), Titan embeddings ingestion	Lambda ↔ OpenSearch ↔ DynamoDB wiring	Case dashboard UI, confirmation screen, TTS playback	Deploy to Polygon Amoy
13–19h	Confidence scoring logic, escalation trigger, prompt hardening	OTP consent endpoints, Cedar policy wiring	Offline PWA shell, language toggle, waveform UI	End-to-end integration testing, CloudWatch logging
20–24h	Polish prompts, prep demo script	Bug fixes, load a clean demo dataset	Final UI polish, responsive check	Architecture diagram slide, demo video recording